

DETAILED PROJECT DOCUMENTATION

Intelligent Freelance Marketplace Platform

Review 1 Documentation — Problem Understanding, Research & Initial DSA Design

Field	Details
Course	Data Structure and Algorithms - II
Course Code	CCSE0301
Programme / Branch	B.Tech CSE-A
Assignment Type	Individual Assignment / PBL
Student	Amulya Pratap Singh
ERP / Roll No.	0251CSE372 / 2501330100061
Project	Intelligent Freelance Marketplace Platform
SDG	SDG 8 — Decent Work and Economic Growth
Review	Review 1 — Problem Understanding & Initial Progress
Review Coverage	Unit/Module 1 (Trees) and Unit/Module 2 (Graphs)
Current Progress	25%
Status	Research, problem analysis, DSA applicability study and conceptual design; implementation not yet started

Note: This document is intentionally more detailed than the 3-page progress report. It is designed to serve as supporting evidence for the work reported in Review 1. The distinction between completed work and proposed/future implementation is maintained throughout.

Table of Contents

1. Introduction

1. Introduction

Freelance marketplaces connect clients who need work completed with independent professionals who offer different technical and creative skills. As the number of projects and freelancer profiles grows, the central challenge is no longer only storing the records; it is efficiently finding and prioritizing the most appropriate candidates for a particular project.

The proposed Intelligent Freelance Marketplace Platform studies this problem from a Data Structures and Algorithms perspective. Instead of treating the platform as only a conventional marketplace containing profiles, projects and CRUD operations, the project focuses on the algorithmic layer that can search candidate records, model relationships among projects and skills, calculate a matching score, rank candidates and return a Top-K recommendation list.

The work completed for Review 1 is primarily analytical and conceptual. It includes understanding the problem, studying existing marketplace/recommendation approaches, reviewing three relevant research papers, identifying a project-specific research gap, studying DSA-II Units 1 and 2, evaluating which structures are genuinely useful, and designing the initial matching workflow. No claim is made that the final matching engine has already been implemented.

The documentation therefore answers four questions:

- Why is freelancer-project matching a meaningful problem?
- What did the literature and existing systems show?
- Why were particular DSA structures selected or rejected?
- How will the selected structures fit together when implementation begins?

2. Project Domain and Context

2.1 Domain

Domain: Intelligent freelance marketplaces and algorithmic freelancer-project recommendation.

2.2 Why this domain was selected

The domain naturally contains several types of records and relationships. A project has requirements, a freelancer has skills and experience, and skills can be related to one another. A client may also specify budget, availability expectations and other constraints. Therefore, the domain provides a practical setting in which multiple DSA operations can be demonstrated without artificially adding unrelated structures.

Entity	Important attributes	Why it matters
Freelancer	ID, skills, experience, rating, availability, rate, project history	Candidate profile used for matching and ranking
Project	ID, required skills, description, budget, deadline, required availability	Source of requirements against which candidates are evaluated
Skill	Skill name, category/domain, related skills	Connects project requirements with freelancer capabilities
Project History	Project ID, role, skills used, outcome	Provides evidence of prior relevant work
Client	Client ID, projects, budget preferences	Creates project requirements and receives recommendations

3. Problem Statement

The proposed platform addresses the problem of efficiently identifying suitable freelancers for a project when many candidate profiles are available. Existing marketplaces already provide search, filtering and recommendation

features, so the project's problem is not simply the absence of search. The focus is on studying how multiple project and freelancer attributes can be organized and processed so that candidate retrieval and ranking become systematic and algorithmically efficient.

The problem can be expressed as:

Given:
A project P with required skills, budget, deadline and other constraints
A collection of freelancer profiles F1...Fn
Determine:
1. Which freelancers satisfy the basic requirements?
2. How strongly does each candidate match the project?
3. How can candidates be ranked efficiently?
4. How can the best K candidates be retrieved without sorting every record repeatedly?
Output:
A ranked Top-K list of suitable freelancers with an explainable matching score.

3.1 Why manual selection becomes difficult

- A client may need to inspect many profiles before finding candidates with the right combination of skills.
- Exact keyword matching can miss semantically related skills or broader skill domains.
- A candidate may have the correct skill but insufficient experience, poor availability or an incompatible rate.
- Multiple factors need to be considered together rather than independently.
- As records grow, repeatedly scanning all profiles becomes less attractive from a computational-efficiency perspective.
- Relationships among skills, projects and previous work are not naturally represented by a simple flat list.

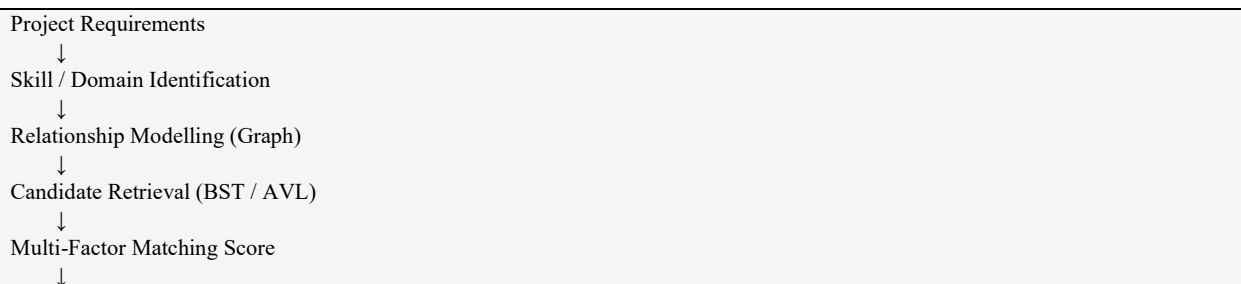
4. Problem Understanding and Identification

4.1 Understanding developed during Review 1

Area	Understanding
Existing problem	Candidate selection can require comparison across skills, experience, rating, availability, history and budget even when search/recommendation tools exist.
Users affected	Clients, freelancers, project managers and platform administrators.
Core information	Skills, requirements, experience, rating, availability, project history, rate, budget and relationships.
Core operations	Search, filter, match, score, rank and Top-K retrieval.
Main DSA need	Ordered search, relationship modelling and priority-based retrieval.
Research direction	Extend skill/domain mapping toward multi-factor freelancer selection and efficient Top-K recommendation.

4.2 Problem formulation

The project treats freelancer recommendation as a pipeline rather than as one algorithm. First, the project requirements are interpreted as skills or domains. Next, potentially relevant candidates are retrieved. Candidates are then evaluated using multiple attributes. Finally, the scores are ranked and the highest-scoring candidates are returned.



Candidate Prioritization (Max Heap)



Top-K Freelancer Recommendations

5. Existing Marketplace and Research Background

During Review 1, the project was deliberately compared conceptually with established freelance marketplace functionality. The purpose was not to claim that existing platforms are simple or lack recommendation systems. Instead, the comparison helped define what is already common in the marketplace domain and where this academic project can focus its DSA contribution.

Aspect	Observed marketplace direction	Project focus
Search	Keyword/category and filter-based discovery	Efficient candidate retrieval using selected DSA structures
Recommendations	Personalized/relevance-based recommendations exist	Explicit multi-factor matching score and Top-K ranking
Profiles	Skills, experience, ratings and other profile information	Use several attributes together in matching
Project requirements	Project briefs/job descriptions are used to identify relevant talent	Project → skill/domain → candidate pipeline
Relationships	Skill/profile relationships can support matching	Graph-based project-skill-freelancer model
Academic contribution	Commercial systems use complex proprietary approaches	Transparent, explainable DSA-based implementation suitable for academic evaluation

Note: The purpose of this comparison is scope definition. The project does not claim to reproduce or outperform commercial platforms; it demonstrates a transparent algorithmic design suitable for the course.

6. Literature Review — Three Selected Papers

Only the three papers discussed for Review 1 are included in this documentation.

6.1 Skill Based Profile Mapping System for Project Assistance Using Machine Learning (2025)

This paper is the closest conceptual foundation to the proposed project. Its central idea is to map project requirements toward technical domains/skills and then identify relevant profiles. It discusses recommendation approaches such as content-based, collaborative and hybrid recommendation and demonstrates the usefulness of domain-level mapping rather than relying only on exact profile wording.

Relevance to our project: It supports the project requirement → skill/domain → profile direction. Our proposed extension is to evaluate multiple freelancer attributes after candidate identification and produce a ranked Top-K list.

6.2 APAED: Time-Optimized Adaptive Parameter Exponential Decay Algorithm for Crowdsourcing Task Recommendation (2025)

This paper studies recommendation in a crowdsourcing environment where workers receive task recommendations. Its contribution is a competition-aware adaptive ranking mechanism using base relevance scores and additional global/local competition information.

Relevance to our project: It demonstrates that recommendation quality can depend on more than static relevance. However, its main direction is worker → task, whereas our project focuses primarily on client/project → freelancer. Therefore it is supporting literature rather than the core paper.

6.3 A Systematic Literature Review on Task Recommendation Systems for Crowdsourced Software Engineering (2025)

This systematic review examines task recommendation research and identifies recurring issues involving datasets, recommendation approaches, generalizability, evaluation and human factors. It emphasizes the need for richer data and more comprehensive evaluation rather than relying on narrow experimental settings.

Relevance to our project: It supports the need to think beyond a single matching factor and to plan measurable evaluation. It also provides a broader research context for recommendation-system design.

6.4 Paper-to-project comparison

Paper	Recommendation direction / focus	What we take from it
Skill Based Profile Mapping	Project/requirements → skills/domains → profiles	Core foundation for skill/domain matching
APAED	Workers → tasks; competition-aware ranking	Recommendation/ranking ideas; direction differs from ours
Systematic Literature Review	Review of task recommendation research	Evidence about dataset, approach, generalizability and evaluation gaps

7. Research Gaps Identified

The gaps below are stated carefully. Where a point comes directly from the reviewed literature, it is presented as a literature observation. Where the project proposes an extension based on the scope of a paper, it is explicitly treated as the project's proposed direction rather than an author-stated limitation.

Gap / Observation	Basis	How the project responds
Skill/domain matching alone may not capture the full selection problem	Inferred from the scope of the skill-based profile-mapping paper	Extend matching with experience, rating, availability, history and budget/rate compatibility
Recommendation direction differs across literature	APAED focuses on worker-to-task recommendation	Study project/client-to-freelancer recommendation as the primary direction
Dataset limitations can reduce recommendation quality	Systematic review	Design a richer attribute model and plan evaluation on multiple matching factors
Approaches may have limited generalizability	Systematic review	Keep the matching pipeline transparent and modular so factors/weights can be tested
Evaluation can be restricted or insufficient	Systematic review; APAED also discusses evaluation limitations	Plan ranking-quality and performance evaluation rather than only demonstrating output
Efficient DSA-based retrieval and Top-K is a project design need	Derived from the project's required operations, not a direct paper claim	Use BST/AVL for retrieval, Graph for relationships and Max Heap for Top-K ranking

7.1 Core project gap statement

The project explores how skill/domain-based project-to-profile matching can be extended into a transparent multi-factor freelancer selection pipeline that efficiently retrieves candidates, models skill relationships and produces Top-K recommendations using appropriate data structures.

7.2 Important viva clarification

The research papers do not specifically prescribe AVL trees, adjacency lists or Max Heaps for this project. Those structures are selected by the project based on the required operations. This distinction is important: literature motivates the recommendation problem and research direction, while DSA analysis determines the implementation structures.

8. Project Objectives

1. Understand the project-to-freelancer matching problem and its practical challenges.
2. Study existing marketplace and recommendation approaches and identify relevant research gaps.

3. Explore multi-factor matching using skills, experience, rating, availability, project history and budget/rate compatibility.
4. Study DSA-II Unit 1 (Trees) and Unit 2 (Graphs) and determine which concepts are genuinely applicable.
5. Use BST/AVL concepts for efficient ordered search and indexing of freelancer/project records.
6. Use Graphs and an Adjacency List to model relationships among projects, freelancers, skills and requirements.
7. Use a Max Heap to prioritize candidates by matching score and support Top-K recommendations.
8. Prepare the conceptual and algorithmic foundation required for implementation, testing and evaluation in later reviews.

9. Stakeholders and User Requirements

Stakeholder	Need	Expected system support
Clients	Find suitable freelancers without manually comparing a large candidate set	Project requirement analysis, candidate filtering, ranking and Top-K recommendations
Freelancers	Receive projects relevant to their skills and profile	Project recommendation based on skills, experience, availability and other attributes
Project Managers	Find individuals or combinations of freelancers for multi-skill work	Relationship-aware candidate discovery and complementary skill analysis
Platform Administrators	Maintain marketplace data and understand recommendation performance	Structured records, monitoring and performance metrics

9.1 Client-side requirement flow

9. Client provides project title/description and required skills.
10. System identifies or indexes relevant skills/domains.
11. Basic constraints such as budget and availability are applied.
12. Potential candidates are retrieved.
13. Candidates receive a matching score.
14. Highest-ranked candidates are returned as Top-K recommendations.

10. Functional and Data Requirements

10.1 Functional requirements

- Accept project requirements.
- Represent freelancer profiles and project records.
- Identify required and available skills.
- Retrieve potentially relevant freelancers.
- Filter candidates using project constraints.
- Calculate a multi-factor matching score.
- Rank candidates by score.
- Return Top-K recommendations.
- Provide an explanation of major score components.
- Measure algorithmic performance during later implementation.

10.2 Proposed freelancer record

```
Freelancer {
  id
  name
  skills[]
  experience
  rating
  availability
  hourlyRate
  projectHistory[]
}
```

10.3 Proposed project record

```
Project {  
  id  
  title  
  description  
  requiredSkills[]  
  budget  
  deadline  
  requiredAvailability  
}
```

10.4 Why these attributes were selected

Attribute	Purpose in matching
Skills	Primary compatibility between project requirements and freelancer capabilities
Experience	Distinguishes candidates with different levels of practical exposure
Rating	Represents historical client feedback as one ranking factor
Availability	Checks whether the freelancer can work within the required period
Project history	Provides evidence of previous relevant work
Rate / Budget	Checks financial compatibility between project and freelancer

11. DSA-II Concepts Studied, Selection, Implementation and Rejection Analysis

This section explains exactly how the DSA-II concepts are connected to the Intelligent Freelance Marketplace Platform. The purpose is not to include every DSA topic simply because it appears in the syllabus. Each structure is evaluated against a concrete system operation: storing/indexing records, searching candidates, modelling relationships, exploring related entities, calculating scores, or producing the final Top-K recommendation.

Important Review 1 status: the concepts in this section were studied, analysed, selected and mapped to the proposed architecture. The complete DSA modules have NOT yet been claimed as implemented in the working application. Coding, integration, testing and measured performance are planned for Review 2. Therefore, the word “implementation” below means the planned technical implementation and explains how it will be carried out.

Overall proposed pipeline: Project Requirements → Required Skills → Graph Candidate Discovery → AVL/BST Indexed Retrieval → Constraint Filtering → Multi-factor MatchScore → Max Heap → Top-K Freelancer Recommendations.

11.1 DSA Selection Philosophy

The project requires three major algorithmic capabilities. First, the system must retrieve relevant freelancer records efficiently. Second, it must represent relationships between projects, skills and freelancers because one entity can be related to many other entities. Third, after calculating a score for each suitable candidate, it must efficiently identify the highest-scoring freelancers.

Based on these operations, tree structures are considered for ordered indexing, graphs for relationship modelling, and heaps for priority-based ranking. Structures whose primary advantages do not correspond to a core requirement are documented as limited, alternative or future concepts rather than being artificially inserted into the design.

11.2 Data That the DSA Structures Will Process

The main records are Freelancer, Project and Skill. A Freelancer contains an ID, skills, experience, rating, availability, rate and project history. A Project contains a project ID, description, required skills, budget, deadline and availability expectations. A Skill contains a normalized skill name, category/domain and related skills.

The same underlying data can therefore be viewed in different ways. The AVL/BST provides an ordered index over a chosen key, the graph represents relationships between entities, and the Max Heap temporarily organizes candidates by their computed MatchScore.

11.3 Binary Tree — Why It Is Studied but Not Implemented as a Core Structure

What it is: A Binary Tree is a hierarchical structure in which each node has at most two children. Unlike a BST, a normal Binary Tree does not require values in the left subtree to be smaller or values in the right subtree to be larger.

Why it is relevant academically: Binary Tree is the foundation for understanding BST, AVL and several other tree structures. Studying it is necessary before selecting a more specialized tree.

Why it is not used for the core marketplace search: The project needs to locate records by an ordered key. A normal Binary Tree has no ordering rule, so a search may require visiting nodes across the tree. In the worst case this is $O(n)$. That does not provide the directed search behaviour required for a growing freelancer index.

Example: Suppose five freelancer records F101, F102, F103, F104 and F105 are stored without any ordering property. To find F105, the system cannot decide from the current node whether it should move left or right. It may have to inspect many nodes.

Possible but non-core use: A Binary Tree could represent a simple hierarchy or decision tree, but the current project does not require such a hierarchy as its primary data structure.

Final decision: Studied for DSA understanding, but NOT implemented as a core project structure because BST/AVL provide the required ordered-search behaviour.

11.4 Binary Search Tree (BST) — Baseline Implementation

What it is: A BST is a Binary Tree with the ordering rule: keys smaller than the current node are stored in the left subtree and keys larger than the current node are stored in the right subtree.

Why we are implementing/studying it: BST directly solves the ordered-search problem. It also provides the baseline against which the project can explain why AVL balancing is useful. The BST therefore demonstrates the basic tree operations before the balanced version is introduced.

How the freelancer index will be represented: Each node will contain an ordered key and the corresponding Freelancer record. The exact final key can be a freelancer ID or another normalized index key; this will be finalized when the data model is implemented.

Conceptual C++ structure:

```
struct Node {  
    string key;  
    Freelancer data;  
    Node* left;  
    Node* right;  
};
```

Insertion implementation: Start from the root. Compare the new freelancer's key with the current node. If the new key is smaller, recursively/iteratively move to the left child. If larger, move to the right child. When a NULL position is reached, create the new node there.

Search implementation: Start at the root. If the current key equals the target, return the record. If the target is smaller, search the left subtree; if larger, search the right subtree. The ordering rule avoids examining the opposite subtree.

Deletion implementation: Three cases are handled. Case 1: the node is a leaf, so it can simply be removed. Case 2: the node has one child, so the parent is connected directly to that child. Case 3: the node has two children, so the node can be replaced by its inorder successor or predecessor and then the replacement node is removed.

Traversal implementation: Inorder traversal visits Left → Root → Right. Because of the BST ordering property, this produces freelancer keys in sorted order.

Concrete example: Insert F103, F101, F105, F100 and F102. F103 becomes the root. F101 goes left, F105 goes right, F100 goes left of F101 and F102 goes right of F101. A search for F102 follows F103 → F101 → F102 rather than scanning every record.

Important limitation: If records are inserted in already sorted order F101, F102, F103, F104, F105, the BST can become a chain. Search, insertion and deletion can then degrade from average $O(\log n)$ behaviour to $O(n)$.

Final decision: BST is selected as the baseline ordered-tree implementation/concept. AVL is preferred when predictable performance is required.

11.5 AVL Tree — Preferred Balanced Tree Implementation

What it is: An AVL Tree is a self-balancing BST. It preserves the BST ordering property but additionally maintains balance so that one side of the tree does not become excessively deeper than the other.

Why we are implementing it: The marketplace is expected to contain an increasing number of freelancer and project records. A normal BST can become skewed depending on insertion order. AVL prevents this problem and guarantees $O(\log n)$ search, insertion and deletion.

Balance factor: Balance Factor = $\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$. For an AVL tree, the balance factor of every node must remain -1 , 0 or $+1$ after rebalancing.

How insertion will work: (1) Perform normal BST insertion. (2) Update the height of every node on the return path. (3) Calculate the balance factor. (4) Detect an imbalance. (5) Apply the required rotation. (6) Return the new balanced root.

How deletion will work: (1) Perform standard BST deletion. (2) Update node heights while returning upward. (3) Recalculate balance factors. (4) Apply rotations wherever necessary. (5) Return the balanced tree.

Rotation cases: LL imbalance $\rightarrow$ right rotation. RR imbalance $\rightarrow$ left rotation. LR imbalance $\rightarrow$ left rotation on the child followed by right rotation. RL imbalance $\rightarrow$ right rotation on the child followed by left rotation.

Example: If keys F101, F102, F103, F104 and F105 are inserted in increasing order, an ordinary BST can become a long chain. AVL performs rotations during insertion so the height remains logarithmic instead of growing linearly.

How it fits the marketplace: When a freelancer profile is added, the profile will be inserted into the AVL index using the selected key. When a profile is searched, the AVL ordering directs the search. When a profile is removed or updated in a way requiring reinsertion, the AVL tree maintains its balance.

Why both BST and AVL are retained in the documentation: BST demonstrates the fundamental ordered-tree solution and its possible degradation; AVL demonstrates the balanced solution selected for predictable performance. This gives a clear DSA comparison instead of treating them as unrelated structures.

Final decision: AVL is a CORE planned implementation for efficient ordered indexing.

11.6 Min Heap — Why It Is NOT the Main Ranking Structure

What it is: A Min Heap is a complete binary tree in which the smallest key has priority and appears at the root.

Why it was studied: Heap structures are useful whenever the system repeatedly needs a minimum or maximum priority element. This makes them relevant to marketplace ranking, but the direction of priority must match the actual requirement.

Possible implementation: A Min Heap could store freelancer rates and quickly expose the lowest-rate candidate. For example, if F101=₹500/hour, F102=₹300/hour, F103=₹700/hour and F104=₹400/hour, F102 would have the minimum rate.

Why it is not selected as the main recommendation heap: The project is not trying to recommend simply the cheapest freelancer. A candidate may have a higher rate but a much stronger skill match, better experience, rating, availability and relevant project history. The final objective is therefore maximum MatchScore, not minimum cost.

Where it could still be useful: A future version could use a Min Heap for budget-oriented queries such as “show the lowest-cost suitable candidates” or for deadline/priority policies where smaller values have higher priority.

Final decision: Studied and documented as LIMITED/OPTIONAL; not the core Top-K ranking structure.

11.7 Max Heap — Core Top-K Recommendation Implementation

What it is: A Max Heap is a complete binary tree where the largest key is at the root. It is designed for repeated access to the highest-priority element.

Why we are implementing it: After filtering, the system will calculate a MatchScore for each suitable freelancer. The final result should be Top-K, for example Top-5 or Top-10 candidates. A Max Heap directly supports repeated extraction of the highest score.

How it will be implemented: Each candidate will be represented as a pair such as (MatchScore, FreelancerID). The pair is inserted into a Max Heap. The heap maintains the largest score at the root. The system then calls `extractMax()` repeatedly K times and stores the returned candidates.

Example scores: F101=91, F102=74, F103=88, F104=95, F105=81. The first extractMax() returns F104 with score 95. The next returns F101 with 91. The third returns F103 with 88. Therefore Top-3 = F104, F101, F103.

Why not just sort every candidate: Full sorting is possible and may be used as a baseline, but Max Heap naturally expresses the priority-queue operation required by repeated “best candidate” extraction. This also gives a direct DSA-II application for the recommendation stage.

Complexity: Heap maximum/peek is $O(1)$; insertion is $O(\log n)$; extractMax is $O(\log n)$. Building a heap from n candidates can be $O(n)$ with a bottom-up heap construction, depending on implementation.

Final decision: Max Heap is a CORE planned implementation for candidate ranking and Top-K extraction.

11.8 Tree Traversal — Supporting Implementation

Traversal is required to systematically process the selected tree structures even though traversal itself is not the recommendation algorithm.

Inorder traversal: Left → Root → Right. In a BST/AVL this returns records in sorted key order. It can be used to inspect the index, generate ordered output or verify that the tree property is maintained.

Preorder traversal: Root → Left → Right. It can be used for structural inspection or serialization-style processing.

Postorder traversal: Left → Right → Root. It is useful for bottom-up processing and can support safe tree cleanup.

Level-order traversal: Processes nodes level by level using a queue. It is useful for observing tree height and balance structure.

Complexity: A complete traversal visits each relevant node once, so the time complexity is $O(n)$.

Final decision: SUPPORTING implementation/concept because tree operations and verification depend on traversal.

11.9 Threaded Binary Tree — Detailed Reason for Rejection

What it is: A Threaded Binary Tree uses otherwise-null child pointers to store links to an inorder predecessor or successor. This can make inorder traversal possible with reduced dependence on recursion or an explicit auxiliary stack.

Why it was considered: The concept is relevant to DSA-II because it improves a specific traversal scenario.

Why it is not implemented in this project: The marketplace's dominant operations are indexed search, graph relationship discovery and score-based ranking. The system does not have a requirement for extremely frequent stack-free inorder traversal. Standard AVL/BST traversal is sufficient and simpler to maintain.

Adding a Threaded Binary Tree would therefore increase implementation complexity without solving a major project requirement. This is an example of intentionally rejecting a syllabus structure when its benefit does not justify its complexity.

Final decision: NOT implemented as a core structure.

11.10 Graph — Core Relationship Model

Why a graph is necessary: The relationship structure of the marketplace is many-to-many. A freelancer can have many skills, one skill can belong to many freelancers, a project can require many skills, and skills can have relationships with other skills. A tree cannot naturally represent all of these cross-connections because a tree is constrained by a hierarchical parent-child structure.

Graph entities: Freelancer, Project and Skill are proposed as vertices.

Graph relationships: HAS_SKILL connects Freelancer → Skill; REQUIRES connects Project → Skill; WORKED_ON connects Freelancer → Project; RELATED_TO connects Skill → Skill.

Example:

```
P201 —REQUIRES—> React
P201 —REQUIRES—> Node.js
F101 —HAS_SKILL—> React
F101 —HAS_SKILL—> Node.js
F101 —WORKED_ON—> P150
React —RELATED_TO—> Next.js
```

How implementation will work: When a project is created, its required skills become relationships from the project node to skill nodes. When freelancer profiles are registered, their skills create HAS_SKILL edges. Previous projects create WORKED_ON relationships. The graph can then be traversed from a project's required skills toward connected freelancer nodes.

How this helps matching: Suppose P201 requires React and Node.js. The graph can discover freelancers connected to those skill nodes. Those discovered candidates are then passed to the filtering and scoring stages. The graph therefore performs relationship-based candidate discovery rather than final score ranking.

Why graph is different from AVL: AVL answers an ordered-index question such as “find this key efficiently.” The graph answers a relationship question such as “which freelancers are connected to the skills required by this project?” Both solve different problems and therefore complement each other.

Final decision: Graph is a CORE planned implementation.

11.11 Adjacency List — Core Graph Representation

An adjacency list stores, for each vertex, a collection of the vertices directly connected to it. It is the preferred representation because the expected marketplace graph is sparse: each entity will be related to only a subset of all possible entities.

Conceptual representation:

```
adjacencyList[P201] = {React, Node.js}
adjacencyList[React] = {P201, F101, F103, Next.js}
adjacencyList[F101] = {React, Node.js, P150}
```

Implementation approach: In C++, an `unordered_map` or `map` can associate a node identifier with a vector/list/set of adjacent node identifiers. When an edge is added, the destination is inserted into the source's adjacency collection. For an undirected relationship, the reverse edge is also inserted; for directed relationships, only the intended direction is stored.

Why it fits the project: The number of possible pairs among V entities can be very large, but only existing relationships need to be stored. The adjacency list therefore avoids allocating space for nonexistent edges.

Complexity: Storage is $O(V+E)$. BFS and DFS using the adjacency list are $O(V+E)$, assuming standard adjacency iteration.

Final decision: Adjacency List is the CORE graph representation.

11.12 Adjacency Matrix — Why It Is Not Preferred

What it is: An adjacency matrix stores a $V \times V$ table. An entry indicates whether an edge exists between two vertices.

Advantage: Direct edge existence checking can be $O(1)$, which is useful for dense graphs with many relationships.

Why it is not preferred here: A marketplace containing many freelancers, skills and projects can have a very large number of possible vertex pairs, while only a relatively small number of relationships may actually exist. An adjacency matrix would still reserve $O(V^2)$ space.

Example: If there are 10,000 graph vertices, a matrix conceptually contains 100 million possible positions even if only a small fraction represent real relationships. An adjacency list stores the actual edges instead.

Why we are not implementing it as the primary structure: The expected graph is relationship-sparse and traversal is more important than constant-time edge lookup for every possible pair. Adjacency List is therefore more space-efficient and better aligned with BFS/DFS.

Could it be used later: Yes. A matrix could be useful if the final system develops a dense subgraph or requires extremely frequent direct edge checks. It can also be included as a performance comparison in a later experiment.

Final decision: NOT preferred as the main implementation; retained as an alternative for comparison.

11.13 BFS — Supporting Graph Search

Breadth-First Search explores a graph level by level using a queue. It is useful when the system wants to discover nearby relationship layers from a starting project or skill.

Implementation: Create an empty queue and a visited set/array. Mark the starting vertex visited and push it into the queue. While the queue is not empty, remove the front vertex, process it, and push each unvisited adjacent vertex while marking it visited.

Marketplace example: Start at project P201, visit its required skills, then visit freelancers connected to those skills. If related skills are allowed, the search can explore an additional relationship layer.

Why BFS is not the ranking algorithm: BFS tells us which entities are reachable and in what traversal order; it does not determine whether F101 is a better freelancer than F103 using rating, experience, budget and history. Those decisions are handled by MatchScore and Max Heap.

Complexity using adjacency list: $O(V+E)$.

Final decision: SUPPORTING algorithm for graph-based candidate discovery.

11.14 DFS — Supporting Graph Search

Depth-First Search explores one branch deeply before backtracking. It can be used to explore connected skills, project histories or relationship components.

Implementation: DFS can be written recursively or with an explicit stack. At a node, mark it visited, process it, and then visit each unvisited adjacent node.

Marketplace example: Starting from a skill such as React, DFS can explore related skills such as Next.js and then discover connected freelancer/project relationships. This can help study the connected skill neighbourhood.

Why DFS is not the final matching algorithm: Like BFS, DFS provides graph exploration rather than multi-factor ranking. It is therefore a supporting operation.

Complexity using adjacency list: $O(V+E)$.

Final decision: SUPPORTING algorithm; useful for relationship exploration and graph analysis.

11.15 Single-Threaded Execution — Initial Implementation Choice

The first implementation will be single-threaded. This means tree updates, graph traversal, score calculation and heap operations are executed in a controlled sequential flow.

Why: The project is still establishing correctness. Running sequentially makes it easier to test each DSA operation independently, reproduce results, detect bugs and measure operation time without concurrency interference.

Implementation approach: A recommendation request will pass through the pipeline in a predictable order: candidate discovery → retrieval/filtering → scoring → heap insertion → Top-K extraction.

Why this is a deliberate decision rather than a missing feature: Multithreading is useful only after the core algorithm is correct. Introducing concurrent access too early could create synchronization and shared-state issues that distract from demonstrating the DSA concepts.

Final decision: SELECTED for the initial implementation.

11.16 Multithreading — Future Enhancement, Not Initial Implementation

Multithreading could later allow multiple recommendation requests to execute concurrently or allow independent candidate scoring operations to run in parallel.

Potential future implementation: Candidate sets could be partitioned into batches, each batch scored by a worker thread, followed by a merge/heap stage. Care would be required if multiple threads update shared data structures.

Why it is not implemented initially: The core project requirement is to demonstrate correct DSA-based searching, relationship modelling and ranking. Concurrency adds synchronization, race-condition and debugging complexity. It is therefore more appropriate as a performance enhancement after the sequential system is stable.

Final decision: FUTURE enhancement, not part of the Review 1 core implementation.

11.17 Multi-Factor MatchScore — How DSA Connects to Recommendation

The DSA structures alone do not decide whether a freelancer is suitable. The recommendation layer first computes a score from project and freelancer attributes. The initial dimensions are Skill Match, Experience, Rating, Availability, Project History and Budget/Rate Compatibility.

Conceptual formula:

$$\text{MatchScore} = w_1 \times \text{SkillMatch} + w_2 \times \text{Experience} + w_3 \times \text{Rating} + w_4 \times \text{Availability} + w_5 \times \text{History} + w_6 \times \text{BudgetCompatibility}$$

Each factor is normalized to a common scale before combining them. The exact weights are intentionally not treated as final in Review 1 because they need to be validated experimentally.

Example: Suppose F101 has complete skill coverage, 3 years' experience, rating 4.7, full availability, relevant project history and compatible rate. F103 may have complete skill coverage and rating 4.8 but only part-time availability. The graph may discover both candidates, but the score differentiates them using the additional factors.

DSA connection: Graph discovers relationship-relevant candidates; AVL/BST retrieves indexed records efficiently; the MatchScore function evaluates candidate suitability; Max Heap ranks the resulting scores and extracts Top-K.

11.18 Complete Proposed Implementation Sequence

Phase 1 — Data preparation: Create sample Freelancer, Project and Skill records with realistic attributes.

Phase 2 — Tree module: Implement BST insertion, search, deletion and inorder traversal as the baseline.

Phase 3 — AVL module: Implement height calculation, balance factor, rotations, insertion, deletion and search. Test LL, RR, LR and RL cases.

Phase 4 — Graph module: Create graph nodes for projects, freelancers and skills. Add HAS_SKILL, REQUIRES, WORKED_ON and RELATED_TO edges.

Phase 5 — Adjacency List: Store graph edges efficiently and verify the resulting relationships.

Phase 6 — BFS/DFS: Traverse the graph from a project or skill and collect relevant connected candidates.

Phase 7 — Filtering: Remove candidates that fail hard constraints such as unavailable status or clearly incompatible budget/rate requirements.

Phase 8 — Scoring: Calculate the normalized multi-factor MatchScore for every remaining candidate.

Phase 9 — Max Heap: Insert (MatchScore, Freelancer) pairs and repeatedly extract the maximum.

Phase 10 — Top-K output: Display the best K freelancers together with their score and, eventually, an explanation of why they were recommended.

Phase 11 — Evaluation: Compare BST and AVL search behaviour, graph traversal cost, heap ranking correctness and recommendation generation time as data size increases.

11.19 Why the Selected Structures Work Together

The structures are not competing alternatives for the same operation. AVL/BST handle ordered indexing; Graph handles many-to-many relationships; Adjacency List stores that graph efficiently; BFS/DFS explore relationships; MatchScore converts multiple profile/project attributes into a comparable value; Max Heap handles priority-based Top-K extraction.

This separation makes the architecture easier to explain and test. Each DSA concept has a specific input, operation, output and complexity. It also avoids the common problem of adding a data structure merely to satisfy a syllabus checklist.

11.20 DSA Concepts Not Implemented as Core — Final Justification

Binary Tree: Not core because it lacks the ordered-search property required for efficient indexed retrieval.

Threaded Binary Tree: Not core because stack-free inorder traversal is not a dominant project requirement and standard AVL/BST traversal is sufficient.

Min Heap: Not core because the primary ranking objective is maximum MatchScore rather than minimum rate/cost. It remains a possible optional structure for minimum-value queries.

Adjacency Matrix: Not preferred because the proposed graph is expected to be sparse and a matrix requires $O(V^2)$ space.

Multithreading: Not initially implemented because the first milestone prioritizes correctness, independent DSA testing and reproducible performance measurements. It is retained as a future optimization.

These are deliberate engineering decisions, not omissions. The project documents both selection and rejection so that the DSA mapping remains technically justified.

11.21 Review 1 vs Review 2 Implementation Boundary

Completed in Review 1: problem understanding, existing-platform study, three-paper literature review, research-gap identification, DSA-II concept study, DSA applicability analysis, relationship model, conceptual matching workflow and preliminary scoring dimensions.

Planned for Review 2: actual BST/AVL coding, graph and adjacency-list implementation, BFS/DFS testing, MatchScore implementation, Max Heap implementation, integration into a Top-K pipeline, test cases, performance measurements and evidence screenshots.

No final accuracy, speed or recommendation-quality result is claimed in Review 1 because those values require an implemented system and evaluation data.

12. Detailed Tree-Based Analysis

12.1 Why a tree is considered

A marketplace contains ordered records that may need to be searched by identifiers, rates, experience levels or other indexed keys. A tree can maintain an ordered organization of such records. However, the choice of tree matters because an ordinary binary tree does not provide a search-order property.

12.2 Binary Tree — why it is not the primary structure

A binary tree allows each node to have at most two children, but it does not inherently define how values are ordered. Therefore, searching for a particular freelancer by key may require visiting many nodes, producing $O(n)$ worst-case search. It can still be used for hierarchical or decision-tree representations, but that is not the central operation of this project.

12.3 Binary Search Tree

A BST adds an ordering rule: values smaller than a node are placed in the left subtree and larger values in the right subtree. This makes ordered search possible. In an approximately balanced tree, search, insertion and deletion are $O(\log n)$ on average. If the tree becomes highly skewed, these operations degrade to $O(n)$.

BST insertion idea:

1. Start at the root.
2. Compare the new key with the current node.
3. Move left if smaller; move right if larger.
4. Continue until an empty position is found.
5. Insert the new record there.

12.4 AVL Tree

AVL is a self-balancing BST. After insertion or deletion, the balance factor of affected nodes is checked and rotations are applied when necessary. The purpose is to avoid the $O(n)$ skewed-tree case of an ordinary BST and provide guaranteed $O(\log n)$ search, insertion and deletion.

AVL operation	Purpose in project	Complexity
Search	Find a freelancer/project using an indexed key	$O(\log n)$ guaranteed
Insert	Add a new freelancer/project record	$O(\log n)$ guaranteed
Delete	Remove an inactive/deleted record	$O(\log n)$ guaranteed
Inorder traversal	Retrieve indexed records in sorted order	$O(n)$

12.5 Why both BST and AVL are discussed

BST is important because it is the fundamental ordered-tree concept studied in the syllabus and provides the baseline for understanding balanced search trees. AVL is the stronger implementation choice when predictable performance is required. Studying both also allows a later implementation comparison showing how balancing changes worst-case behavior.

12.6 Threaded Binary Tree

A threaded binary tree replaces selected null pointers with links to inorder predecessor/successor nodes so that inorder traversal can be performed with less dependence on recursion or an explicit stack. This is useful in systems requiring repeated stack-free traversal. The current marketplace concept does not have such a dominant traversal requirement, so it is studied but not selected as a core structure.

12.7 Tree traversal

Traversal	Complexity	Potential role
Inorder	$O(n)$	Sorted output from a BST/AVL index
Preorder	$O(n)$	Serialization or structured export of a tree
Postorder	$O(n)$	Bottom-up processing/deletion-style workflows
Level-order	$O(n)$	Breadth-wise inspection of a tree

13. Detailed Graph-Based Analysis

13.1 Why a graph is necessary

The project contains many-to-many relationships. One freelancer can have many skills; one skill can belong to many freelancers; a project can require many skills; and a skill can be related to other skills. A graph represents these relationships more naturally than a tree because there is no requirement for one unique parent for every entity.

13.2 Proposed graph model

Freelancer — HAS_SKILL —> Skill
Freelancer — WORKED_ON —> Project
Project — REQUIRES —> Skill
Skill — RELATED_TO —> Skill

13.3 Graph components

Component	Representation
Vertex	Freelancer, Project or Skill
Edge	Relationship such as HAS_SKILL, REQUIRES, WORKED_ON or RELATED_TO
Weight (optional)	Relevance strength, similarity, prior project evidence or another matching factor
Adjacency List	For each vertex, store a list of connected vertices/edges

13.4 Adjacency List vs Adjacency Matrix

Aspect	Adjacency List	Adjacency Matrix
Space	$O(V+E)$	$O(V^2)$
Best suited for	Sparse graphs	Dense graphs
Marketplace fit	Preferred because each entity connects to a relatively small subset of all entities	Less preferred because storing every possible pair is wasteful
Edge check	$O(\text{degree})$	$O(1)$
Traversal	$O(V+E)$	$O(V^2)$

13.5 BFS and DFS

BFS can explore relationships level by level, while DFS explores one path deeply before backtracking. In the proposed system, these traversals can support exploration of related skills or project/freelancer relationships. They are supporting graph operations rather than the final ranking algorithm.

Operation	Possible project use	Complexity with adjacency list
-----------	----------------------	--------------------------------

BFS	Explore nearby/related skills or connected entities	$O(V+E)$
DFS	Explore all reachable relationships or connected subgraphs	$O(V+E)$
Connectivity	Check whether a set of entities is connected through defined relationships	$O(V+E)$

14. Proposed Intelligent Matching Method

14.1 Why multi-factor matching

A freelancer with the correct skill is not automatically the best candidate. Two candidates may have similar skills but differ significantly in experience, rating, availability, previous project relevance or rate. Therefore, the proposed matching score combines several measurable factors.

14.2 Proposed score components

Factor	Example interpretation
Skill Match	Percentage of required skills covered by the freelancer
Experience	Normalized relevance of years/level of experience
Rating	Normalized historical rating
Availability	Compatibility with project schedule
Project History	Similarity/relevance of previous work
Budget Compatibility	How well freelancer rate fits project budget

14.3 Proposed weighted scoring model

```
MatchScore =
  w1 * SkillMatch
+ w2 * ExperienceScore
+ w3 * RatingScore
+ w4 * AvailabilityScore
+ w5 * ProjectHistoryScore
+ w6 * BudgetCompatibilityScore
```

where:

$$w1 + w2 + w3 + w4 + w5 + w6 = 1$$

The exact weights will be finalized and tested during implementation.

The weights are intentionally not fixed in Review 1. Final values should be justified through testing rather than invented only for the report. During implementation, several weight configurations can be compared using ranking-quality and performance measures.

14.4 Explainability

The recommendation should not only return a freelancer ID and a numeric score. A later implementation can show the major contributing factors, for example: strong skill match, high relevant experience, compatible availability and acceptable rate. This makes the ranking easier to interpret and evaluate.

15. Proposed End-to-End System Workflow

```
STEP 1 Client submits project requirements
↓
STEP 2 Extract required skills / domains
↓
STEP 3 Use graph relationships to identify relevant entities
↓
STEP 4 Retrieve candidate freelancer records
↓
STEP 5 Apply basic filters (availability, budget, etc.)
```

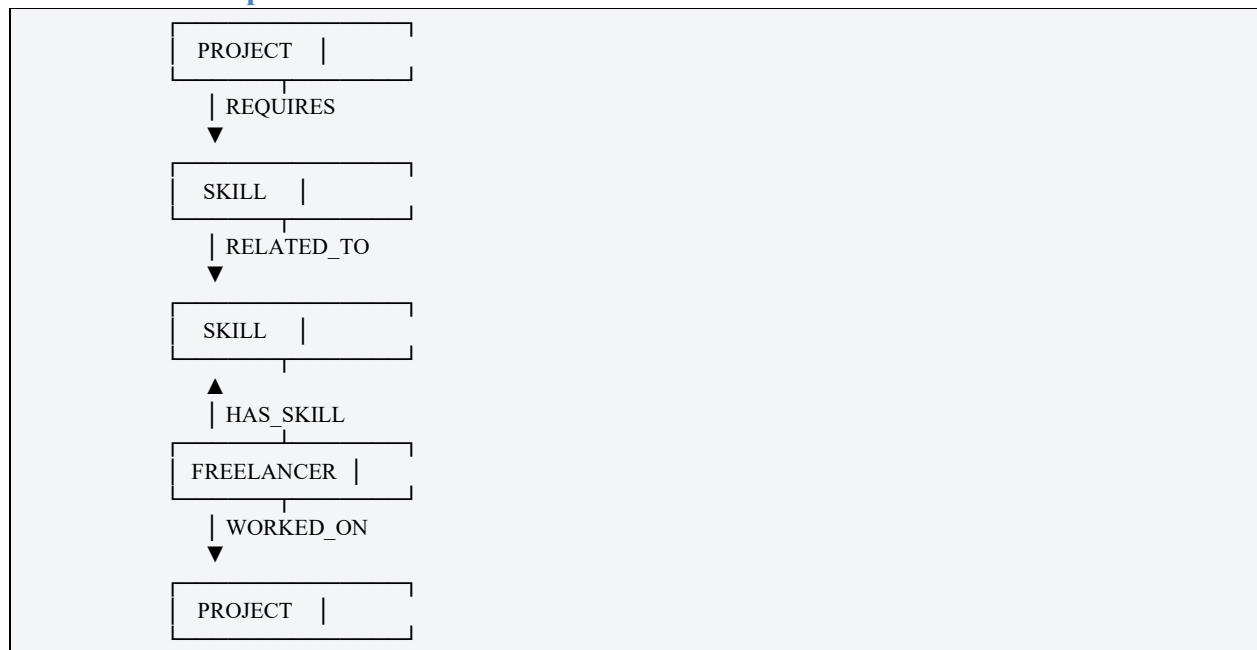
↓	
STEP 6	Calculate multi-factor MatchScore
↓	
STEP 7	Insert / process candidates in Max Heap by score
↓	
STEP 8	Extract highest-scoring K candidates
↓	
STEP 9	Return ranked Top-K recommendations
↓	
STEP 10	Evaluate ranking quality and execution performance

15.1 Why the stages are separated

Stage	Reason for separation
Requirement analysis	Converts unstructured project needs into usable matching attributes
Graph exploration	Captures relationships and related skills
Candidate retrieval	Avoids performing expensive scoring on clearly irrelevant records
Filtering	Removes candidates that fail hard constraints
Scoring	Produces a comparable numerical measure
Heap ranking	Efficiently prioritizes the highest-scoring candidates
Evaluation	Separates correctness/quality from implementation speed

16. Conceptual Data Model

16.1 Relationship model



16.2 Example record set

Freelancer	Skills	Experience	Rating	Availability	Rate
F101	React, Node.js, MongoDB	3 years	4.7	Available	₹800/hr
F102	Java, Spring, SQL	4 years	4.5	Available	₹700/hr
F103	React, Next.js, Node.js	2 years	4.8	Part-time	₹650/hr

For a project requiring React + Node.js, F101 and F103 would be natural candidates. The graph can capture their skill relationships, while the multi-factor score can distinguish them using experience, rating, availability, history and rate.

17. Proposed Algorithms and Pseudocode

17.1 Candidate retrieval using BST/AVL

```
searchCandidate(root, key):  
  if root == NULL:  
    return NOT_FOUND  
  
  if key == root.key:  
    return root  
  
  if key < root.key:  
    return searchCandidate(root.left, key)  
  else:  
    return searchCandidate(root.right, key)
```

In the final implementation, the key could represent an indexed attribute such as freelancer ID, a normalized skill key or another search key. The exact indexing strategy will be finalized after the data model is implemented.

17.2 Graph traversal

```
BFS(start):  
  create queue Q  
  mark start visited  
  push start into Q  
  
  while Q is not empty:  
    u = pop Q  
    process u  
  
    for each v in adjacencyList[u]:  
      if v is not visited:  
        mark v visited  
        push v into Q
```

17.3 Matching score

```
calculateScore(project, freelancer):  
  skill    = skillMatch(project.requiredSkills, freelancer.skills)  
  experience = normalize(freelancer.experience)  
  rating   = normalize(freelancer.rating)  
  available = availabilityMatch(project, freelancer)  
  history  = projectHistoryMatch(project, freelancer)  
  budget   = budgetCompatibility(project.budget, freelancer.rate)  
  
  return w1*skill + w2*experience + w3*rating  
         + w4*available + w5*history + w6*budget
```

17.4 Top-K using Max Heap

```
for each candidate:  
  score = calculateScore(project, candidate)  
  maxHeap.push(score, candidate)  
  
repeat K times:  
  result.append(maxHeap.extractMax())  
  
return result
```

The advantage of a Max Heap is that the highest score is always available at the root. This is directly aligned with the Top-K requirement. If all candidates must be fully sorted, a sorting algorithm may also be considered; however, the heap provides a direct priority-queue abstraction for repeatedly extracting the best candidates.

18. Complexity and Performance Considerations

Component	Typical complexity	Why it matters
BST search	$O(\log n)$ average; $O(n)$ worst	Efficient ordered search but can degrade if unbalanced
AVL search/insert/delete	$O(\log n)$ guaranteed	Predictable performance as records grow
Tree traversal	$O(n)$	Every relevant node may need to be processed
Heap peek	$O(1)$	Immediate access to best candidate
Heap insert	$O(\log n)$	Maintains priority ordering after candidate insertion
Heap extract max	$O(\log n)$	Retrieves next highest-scoring candidate
Graph BFS/DFS	$O(V+E)$	Efficient for sparse relationship graphs
Adjacency List space	$O(V+E)$	Avoids storing all possible vertex pairs
Adjacency Matrix space	$O(V^2)$	Can become expensive as entities grow

18.1 Evaluation planned for later reviews

- Search time comparison between an unbalanced BST and AVL tree.
- Insertion/deletion performance as the number of records increases.
- Graph traversal time as vertices and edges increase.
- Recommendation generation time for different candidate-set sizes.
- Top-K ranking correctness.
- Ranking-quality metrics such as Precision@K/Recall@K or other suitable measures after an evaluation dataset is available.
- Comparison with a simple keyword/filter baseline where feasible.

19. Work Completed During Month 1

The following work is completed and forms the evidence base for Review 1:

- Finalized the project title and problem direction.
- Defined the problem of intelligent freelancer-project matching.
- Identified target stakeholders and the information that must be managed.
- Studied existing marketplace/recommendation approaches to avoid making unrealistic assumptions about current platforms.
- Reviewed exactly three research papers relevant to skill mapping, recommendation/ranking and task-recommendation literature.
- Identified the research gap and distinguished literature-supported observations from project-specific design decisions.
- Studied DSA-II Unit 1 (Trees), including Binary Tree, BST, AVL, Threaded Binary Tree, traversal and heap concepts.
- Studied DSA-II Unit 2 (Graphs), including graph representation, adjacency structures and traversal concepts.
- Evaluated which DSA topics are genuinely useful, which are limited-use and which are not required.
- Designed the initial project-skill-freelancer graph model.
- Designed the conceptual matching workflow from project requirements to Top-K recommendations.
- Defined the initial multi-factor scoring dimensions.
- Prepared the implementation, testing and evaluation direction for Review 2.

19.1 What has NOT been claimed as completed

- The final database has not yet been implemented.
- The AVL/BST/Graph/Heap modules have not yet been integrated into a working application.
- The final matching weights have not yet been experimentally validated.
- No external dataset has yet been used for final model evaluation.
- No final recommendation accuracy or performance result is being claimed in Review 1.

20. Evidence and Supporting Material

The 3-page PBL Progress Report – I is the formal Review 1 summary. This detailed document is intended to sit behind that report as evidence and explain how each reported item was reached.

Evidence item	What it demonstrates
Problem statement and context	Understanding of the marketplace problem and its scale-related difficulty
Three-paper literature review	Research activity and basis for the proposed direction
Research-gap table	Connection between literature observations and project scope
DSA applicability table	Reasoned selection/rejection of syllabus concepts
Graph model	Understanding of project-skill-freelancer relationships
Matching workflow	Conceptual integration of retrieval, scoring and ranking
Pseudocode	Initial understanding of how selected DSA operations will be implemented
Complexity table	Awareness of computational cost and scalability
Review 2 plan	Transition from analysis to implementation and testing

20.1 Evidence placement

For submission, the 3-page progress report can remain the official concise document, while this detailed documentation can be attached or shown as the supporting evidence document. Screenshots of future code, console output, diagrams and test results should be added to this document as implementation progresses.

21. Challenges Faced and How They Were Addressed

Challenge	Why it was difficult	How it was addressed
Relating DSA to a real problem	It was easy to describe a marketplace but harder to identify meaningful algorithmic operations.	Mapped each DSA concept to a specific operation: search, relationship modelling or ranking.
Avoiding forced DSA usage	The syllabus contains multiple structures that are not equally relevant.	Evaluated each topic and explicitly documented No/Limited/Future decisions.
BST vs AVL	Both support ordered search and could appear redundant.	Treated BST as the baseline ordered tree and AVL as the balanced implementation with guaranteed $O(\log n)$.
Tree vs Graph	Both can organize data, but their structures are fundamentally different.	Used trees for ordered/indexed records and graphs for many-to-many relationships.
Min Heap vs Max Heap	Both are priority queues but solve opposite priority directions.	Selected Max Heap for highest matching score; Min Heap remains a limited option for lowest-rate/lowest-cost queries.
Understanding research gaps	A paper's scope is not automatically its stated limitation.	Separated author-stated limitations from project-specific inferences.
Recommendation vs DSA	Recommendation logic and data structures solve different layers of the problem.	Defined the score as recommendation logic and BST/AVL/Graph/Heap as the algorithmic data-management layer.
Designing without overcomplication	Adding every possible AI feature could make the project unrealistic for a DSA course.	Kept the Review 1 core around search, graph relationships, scoring and Top-K ranking.

22. Individual Contribution

As this is an Individual Assignment, all work documented in Review 1 was carried out by me. My contribution includes both research and technical analysis.

- Defined the project problem statement and project scope.
- Studied the freelancer-project matching problem and the difficulty of manual comparison.
- Studied existing marketplace and recommendation approaches.
- Read and analyzed the three selected research papers.

- Identified research observations and formulated the project-specific gap.
- Studied DSA-II Unit 1 and Unit 2 concepts relevant to the project.
- Compared DSA structures using operation complexity and project suitability.
- Designed the project-skill-freelancer graph model.
- Designed the initial multi-factor matching and Top-K recommendation pipeline.
- Prepared the initial pseudocode, complexity analysis and implementation plan.
- Prepared the Review 1 report and this supporting documentation.

23. Current Status and Progress

The reported overall progress is 25%. This percentage represents the completion of the analytical foundation rather than 25% of a fully implemented software product.

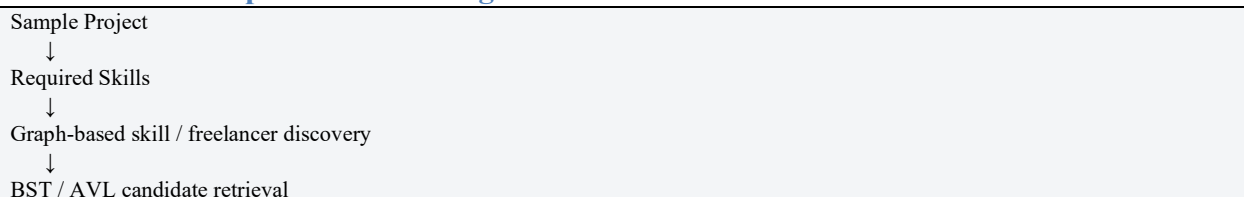
Work area	Status	Evidence in this documentation
Problem understanding	Completed	Sections 3–4
Stakeholder/requirements analysis	Completed	Sections 9–10
Existing marketplace study	Completed at conceptual level	Section 5
Three-paper literature review	Completed	Section 6
Research-gap identification	Completed	Section 7
Unit 1 Trees study	Completed	Sections 11–12
Unit 2 Graphs study	Completed	Sections 11 and 13
Initial DSA mapping	Completed	Sections 11–13
Matching workflow design	Completed conceptually	Sections 14–17
Implementation	Not yet completed	Planned for Review 2
Testing/performance evaluation	Not yet completed	Planned for later reviews

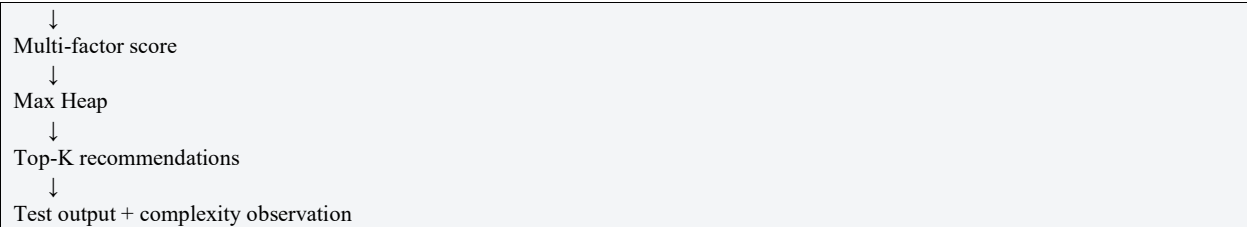
24. Plan for Review 2

The next review moves from conceptual analysis toward implementation. The instruction for the next stage is to continue studying the relevant syllabus material, particularly Units 2 and 3, while understanding how the concepts can actually be applied and implemented.

15. Study relevant topics from DSA-II Units 2 and 3, including their operations, applications and implementation.
16. Determine which Unit 2/3 concepts are practically applicable to the final system.
17. Finalize the data representation for freelancers, projects, skills and graph relationships.
18. Implement a small working version of the selected BST/AVL, graph and heap operations.
19. Create sample freelancer and project records for initial testing.
20. Implement the first version of skill matching and multi-factor scoring.
21. Use a Max Heap to generate initial Top-K recommendations.
22. Test insertion, search, traversal, filtering, scoring and ranking operations.
23. Collect implementation evidence: source-code screenshots, diagrams, sample inputs/outputs and complexity observations.
24. Compare expected and observed behavior and document Review 2 outcomes.

24.1 Review 2 implementation target





25. Conclusion

Review 1 established the conceptual and algorithmic foundation of the Intelligent Freelance Marketplace Platform. The work began by recognizing that modern freelance marketplaces already provide search, filters and recommendation features. The project therefore does not treat the problem as simply “build a freelance website”; instead, it focuses on the underlying algorithmic challenge of efficiently selecting and ranking suitable freelancers from many records.

The literature review of the three selected papers provided three complementary perspectives: skill/domain-based profile mapping, recommendation/ranking under crowdsourcing conditions, and broader evidence about dataset, approach and evaluation limitations in task recommendation research. From these observations, the project proposes extending skill/domain matching toward multi-factor freelancer selection and efficient Top-K recommendation.

From the DSA perspective, the analysis concluded that BST/AVL are relevant for ordered searching, Graphs and Adjacency Lists are relevant for many-to-many skill/project/freelancer relationships, and a Max Heap is a natural structure for prioritizing high matching scores. Binary Tree, Threaded Binary Tree and Adjacency Matrix were not selected as core structures because their strongest use cases do not align with the current primary operations, while Min Heap and multithreading remain limited/future possibilities.

The project is intentionally not presented as implemented at this stage. The next step is to convert the conceptual model into a small working implementation, validate the selected DSA operations, generate Top-K recommendations on sample data and begin collecting measurable evidence for later reviews.

26. References

- Skill Based Profile Mapping System for Project Assistance Using Machine Learning (2025), Journal of Emerging Technologies and Innovative Research (JETIR).
- APAED: Time-Optimized Adaptive Parameter Exponential Decay Algorithm for Crowdsourcing Task Recommendation (2025), Applied Sciences, 15, 9577. DOI: 10.3390/app15179577.
- A Systematic Literature Review on Task Recommendation Systems for Crowdsourced Software Engineering (2025), Information and Software Technology, 184, 107753. DOI: 10.1016/j.infsof.2025.107753.
- Intelligent Freelance Marketplace Platform — PBL Progress Report I, Review 1, Data Structure and Algorithms - II (CCSE0301), prepared for the individual assignment.
- dsa pbl documentation ref.pdf — reference documentation structure supplied for formatting/detail expectations.

Note: Source discipline: The three research papers are used to motivate the literature and research-gap discussion. DSA selections such as AVL, Graph, Adjacency List and Max Heap are project design decisions derived from required operations, not claims that the reviewed papers prescribed those structures.